NoSQL Database Design – Lab Sheet 3

Database: MongoDB

Concise solutions for all 15 questions, with each answer in a separate copyable code block.

1\. Student Activity Tracking System

```javascript
use CollegeDB

db.students.insertOne({
  studentId: 101,
  name: "Amit",
  courses: ["BCA", "Python"],
  attendance: 85,
  skills: ["Java", "MongoDB"],
  activities: [
    { activity: "Quiz", date: new Date(), score: 90 },
    { activity: "Assignment", date: new Date(), score: 80 }
  ]
})

db.students.find({ attendance: { $lt: 75 } })

db.students.find({ skills: "MongoDB" })

db.students.find({ "activities.score": { $gte: 90 } })

db.students.find({ courses: "Python" })
```

2\. Event Logging System

```javascript
db.events.insertMany([
```

```
  { user: "Amit", event: "login" },
  { user: "Amit", event: "failed-login" },
  { user: "Amit", event: "failed-login" },
  { user: "Riya", event: "file-upload" },
  { user: "Riya", event: "logout" },
  { user: "Amit", event: "password-change" }
])
db.events.aggregate([
  { $match: { event: "failed-login" } },
  { group: { \_id: "user", attempts: { $sum: 1 } } },
  { $match: { attempts: { $gte: 2 } } }
])
db.events.aggregate([
  { group: { \_id: "event", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

3\. Mini Blogging Platform

```javascript
db.blogs.insertOne({
  title: "MongoDB Basics",
  author: "Amit",
  category: "Technology",
  tags: ["MongoDB", "Database"],
  comments: [{ user: "Riya", text: "Great post!" }],
```

```
  published: true,
  publishedAt: new Date()
})
db.blogs.find({ tags: "MongoDB" })
db.blogs.find({ category: "Technology" })
db.blogs.find({
  category: "Technology",
  tags: { $all: ["MongoDB", "Database"] }
})
db.blogs.find({ published: true })
```

4\. Flexible Product Catalog

```javascript
db.products.insertMany([
  { name: "Laptop", category: "Laptop", ram: "16GB", storage: "512GB" },
  { name: "Phone", category: "Mobile", battery: "5000mAh", camera: "50MP" },
  { name: "Camera", category: "Camera", resolution: "24MP", zoom: "10x" }
])
db.products.find({ category: "Laptop", ram: "16GB" })
db.products.find({ category: "Mobile", battery: "5000mAh" })
db.products.find({ category: "Camera", resolution: "24MP" })
```

5\. E-Commerce Order Management System

```javascript
db.orders.insertOne({
```

```
  customerId: 101,
  customer: "Amit",
  products: [
    { name: "Mouse", price: 500, quantity: 2 },
    { name: "Keyboard", price: 1000, quantity: 1 }
  ]
})
db.orders.find({ customerId: 101 })
db.orders.aggregate([
  { $match: { customerId: 101 } },
  { $unwind: "products" },
  { $group: {
      _id: "$_id",
      total: { sum: { $multiply: &#91;"products.price", "$products.quantity"] } }
  }}
])
```

6\. Website Analytics System

```javascript
db.visits.insertMany([
  { page: "/home", userId: 1, device: "Mobile", browser: "Chrome", location: "Delhi", time: new Date() },
  { page: "/home", userId: 2, device: "Desktop", browser: "Firefox", location: "Mumbai", time: new Date() },
  { page: "/about", userId: 3, device: "Mobile", browser: "Chrome", location: "Delhi", time: new Date() }
])
db.visits.aggregate([
```

```
  { group: { \_id: "page", visits: { $sum: 1 } } },
  { $sort: { visits: -1 } }
])
db.visits.aggregate([
  { group: { \_id: "device", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

7\. Content Management System

```javascript
db.articles.insertMany([
  { title: "MongoDB", author: "Amit", category: "Tech", tags: ["DB"], status: "published" },
  { title: "Python", author: "Riya", category: "Programming", tags: ["Coding"], status: "draft" }
])
db.articles.find({ status: "published" })
db.articles.find({ status: "draft" })
db.articles.find({ category: "Tech" })
db.articles.find({ tags: "Coding" })
db.articles.find({ author: "Amit", status: "published" })
```

8\. Shopping Cart System

```javascript
db.carts.insertOne({
  userId: 101,
  items: [
```

```
    { product: "Mouse", price: 500, quantity: 2 },
    { product: "Keyboard", price: 1000, quantity: 1 }
  ]
})
// Add product
db.carts.updateOne(
  { userId: 101 },
  { $push: { items: { product: "USB", price: 300, quantity: 1 } } }
)
// Change quantity
db.carts.updateOne(
  { userId: 101, "items.product": "Mouse" },
  { set: { "items..quantity": 3 } }
)
// Remove product
db.carts.updateOne(
  { userId: 101 },
  { $pull: { items: { product: "USB" } } }
)
// Calculate cart value
db.carts.aggregate([
  { $match: { userId: 101 } },
  { unwind : "items" },
  { $group: {
      _id: "$userId",
```

```
    total: { sum: { $multiply: &#91;"items.price", "$items.quantity"] } }
  }}
])
```

9\. Product Review and Rating System

```javascript
db.reviews.insertMany([
  { productId: 1, user: "Amit", rating: 5, review: "Excellent" },
  { productId: 1, user: "Riya", rating: 4, review: "Good" },
  { productId: 2, user: "Raj", rating: 3, review: "Average" }
])
db.reviews.aggregate([
  { $group: {
    _id: "$productId",
    averageRating: { $avg: "$rating" },
    totalReviews: { $sum: 1 }
  }}
])
```

10\. Inventory Monitoring System

```javascript
db.inventory.insertMany([
  { product: "Mouse", stock: 5, minStock: 10, supplier: "ABC", restockDate: new Date() },
  { product: "Keyboard", stock: 20, minStock: 5, supplier: "XYZ", restockDate: new Date() }
])
```

```
db.inventory.find({
  expr: { $lte: &#91;"stock", "$minStock"] }
})
db.inventory.updateOne(
  { product: "Mouse" },
  { $set: { stock: 30, restockDate: new Date() } }
)
```

11\. Bank Wallet Transfer System Using Transactions

```javascript
// Requires MongoDB replica set or sharded cluster
db.wallets.insertMany([
  { userId: 1, balance: 5000 },
  { userId: 2, balance: 2000 }
])
const session = db.getMongo().startSession();
const walletDB = session.getDatabase("CollegeDB");
session.startTransaction();
try {
  walletDB.wallets.updateOne(
    { userId: 1, balance: { $gte: 1000 } },
    { $inc: { balance: -1000 } }
  );
  walletDB.wallets.updateOne(
    { userId: 2 },
```

```
    { $inc: { balance: 1000 } }
  );
  session.commitTransaction();
} catch (e) {
  session.abortTransaction();
  throw e;
} finally {
  session.endSession();
}
```

12\. User Notification System

```javascript
db.notifications.insertMany([
  { userId: 1, message: "New message", type: "message", read: false },
  { userId: 1, message: "Order shipped", type: "order", read: false },
  { userId: 2, message: "System alert", type: "alert", read: true }
])
// Retrieve unread notifications
db.notifications.find({ userId: 1, read: false })
// Mark notification as read
db.notifications.updateOne(
  { userId: 1, message: "New message" },
  { $set: { read: true } }
)
```

13\. Course Learning Platform

```javascript
db.learning.insertMany([
 { student: "Amit", course: "MongoDB", instructor: "Mr. Raj", modules: 10, completed: 8, status: "In Progress" },
 { student: "Riya", course: "Python", instructor: "Ms. Neha", modules: 10, completed: 10, status: "Completed" }
])
// Students with progress above 50%
db.learning.aggregate([
 {
  $match: {
   $expr: {
    $gt: [
     { divide: &#91;"completed", "$modules"] },
     0.5
    ]
   }
  }
 }
])
// Completed courses
db.learning.find({ status: "Completed" })
// Students in a particular course
db.learning.find({ course: "MongoDB" })
```

14\. Real-Time Order Tracking System

```javascript
db.tracking.insertOne({
  orderId: 101,
  status: "Delivered",
  history: [
    { status: "Placed", time: new Date() },
    { status: "Confirmed", time: new Date() },
    { status: "Shipped", time: new Date() },
    { status: "Delivered", time: new Date() }
  ]
})
// Update current status and store history
db.tracking.updateOne(
  { orderId: 101 },
  {
    $set: { status: "Shipped" },
    $push: {
      history: { status: "Shipped", time: new Date() }
    }
  }
)
// Retrieve current status
db.tracking.find(
  { orderId: 101 },
```

```
  { _id: 0, orderId: 1, status: 1 }
)
// Retrieve complete history
db.tracking.find({ orderId: 101 })
```

15\. Flexible News Database

```javascript
db.news.insertMany([
  {
    title: "Technology Update",
    author: "Amit",
    category: "Technology",
    tags: ["AI", "Tech"],
    location: "Delhi",
    images: ["news.jpg"]
  },
  {
    title: "Sports News",
    category: "Sports",
    tags: ["Cricket"],
    sources: ["News Agency"]
  }
])
// Search by category
db.news.find({ category: "Technology" })
```

```
// Search by tag

db.news.find({ tags: "Cricket" })

// Search articles containing a location

db.news.find({ location: { $exists: true } })

// Search regardless of optional fields

db.news.find({

  $or: [

    { author: "Amit" },

    { tags: "AI" },

    { category: "Sports" }

  ]

})

// Search by title

db.news.find({ title: /Technology/i })
```